CTF Challenge: RAZZ SECURITY SYSTEMS v3.7.1-kern

Author: Brijeshzzz / Antigravity **Date:** May 3, 2026 **Category:** Reverse Engineering / Binary Exploitation **Difficulty:** Medium-Hard

---

1. Challenge Overview

RAZZ Security Systems has deployed a prototype kernel monitoring tool. The system appears unstable — threads are behaving unpredictably. The goal is to analyze the binary, understand its internal state, and retrieve the hidden flag.

The flag is **not** stored statically. Multiple layers of anti-debugging and obfuscation are active to prevent trivial extraction.

---

2. Technical Architecture

2.1 Multi-threaded Execution

The binary utilizes three main worker threads and a primary monitor loop:

1. **Mutator Thread**: Incrementally pushes internal state variables (`key_part1`, `key_part2`, `trigger`) toward target values using randomized steps.
2. **Corruptor Thread**: Aggressively sabotages the state. It has 11 different corruption modes (reset, XOR, bit-shift, swap, full-wipe) firing every 2-8ms.
3. **Checker Thread**: Monitors the state. It requires the magic conditions to be met **5 consecutive times** before triggering the flag generation.
4. **Main Thread**: Simulates a kernel monitor with randomized "health checks" and status summaries.

2.2 Anti-Debugging & Integrity Checks

1. **ptrace Check**: Uses `ptrace(PTRACE_TRACEME, 0, NULL, NULL)` in a constructor to detect attached debuggers.
2. **Parent Process Check**: Inspects `/proc/ppid/comm`. If a debugger (gdb, lldb, strace, etc.) is detected, it silently poisons the decoding keys.
3. **Timing Check**: Executes 100,000 iterations of a deterministic LCG. If execution takes longer than 2 seconds, it assumes single-stepping/debugging and poisons the output.

2.3 Flag Encoding System

The flag `flag{razzsecurity-by-brijesh}` is encoded using a per-byte multi-transform system:

- **Algorithm**: `encoded[i] = (plaintext[i] ^ key[i] ^ canary_byte) + offset[i]`
- **Canary Byte**: Derived from the low byte of the LCG result (`0x98`).
- **In-Memory Masking**: Even after decoding, the flag is stored in memory XORed with a unique per-position mask.

---

3. Implementation Details (Source Snippet)

The Core Logic (C)

```c
/* Decoding logic in *decode*chunk */
uint8*t decoded = (uint8*t)((*eflag[i] - off) ^ key ^ *canary*byte);

/* Rolling re-mask ensures full plaintext never exists in RAM */
uint8*t pos*mask = (uint8*t)((i * 37 + 0xAB) ^ (*canary*byte >> 3));
((volatile char *)st->flag)[i] = (char)(decoded ^ pos_mask);
```

---

4. Automated Solve Script (Python)

The following script automatically reconstructs the flag by simulating the LCG and reversing the encoding transformations.

```python
#!/usr/bin/env python3

import sys

def compute*timing*canary():
    x = 0x12345678
    for _ in range(100000):
        x = ((x * 6364136223846793005) + 1442695040888963407) & 0xFFFFFFFFFFFFFFFF
    return x & 0xFF

def main():
    canary*byte = *compute*timing_canary()

    eflag = [
        0xBF, 0x67, 0x0D, 0xC0, 0x9D, 0x4D, 0xD8, 0xAF, 0x34,
        0xF7, 0x99, 0x50, 0x42, 0x99, 0xEA, 0x05, 0xC5, 0x31,
        0xA1, 0xE3, 0x31, 0xCB, 0x1D, 0x30, 0xD9, 0x18, 0x6B,
```

```
    0xB3, 0x4B
]
fbk = [
    0x42, 0x9A, 0xF1, 0x3D, 0x77, 0xBB, 0x2E, 0x55, 0xCC,
    0x11, 0x6F, 0xAA, 0xD3, 0x48, 0x19, 0xE7, 0x5C, 0x83,
    0x64, 0x0B, 0x99, 0x37, 0xFE, 0xC5, 0x2A, 0xDD, 0x8E,
    0x46, 0xA1
]
fbo = [
     3, -7,  5, -2,  9, -4,  1, -8,  6,
    -3,  7, -1,  4, -9,  2, -6,  8, -5,
     3, -7,  5, -2,  9, -4,  1, -8,  6,
    -3,  7
]

flag = ""
for i in range(len(eflag)):
    val = ((eflag[i] - fbo[i]) & 0xFF) ^ fbk[i] ^ canary_byte
    flag += chr(val)

print(f"Decoded Flag: {flag}")
```

if **name** == "**main**": main() ```

---

5. Manual Cracking Steps

1. **Anti-Debug Bypass**: Use `LD_PRELOAD` or `GDB` catch syscalls to bypass `ptrace`.
2. **Canary Extraction**: Analyze the constructor to find the LCG and compute the `0x98` value.
3. **Target Values**: Identify target state values: `0x1337C0DE`, `0xDEADBEEF`, `0x0BADC0DE`.
4. **Memory Extraction**: Pause after the checker triggers and extract `flag[]` and `pos_masks[]` from the `g_state` struct. XOR them to get the plaintext.

Generated by Antigravity CTF Architect